

Problem Statement Description

AI Research Navigator

A Citation-Grounded Companion for AI/ML Learners

Stack mandates: Qdrant (vector store, with metadata filtering), LangGraph (agent orchestration), Python

Submission: GitHub repository + recorded demo + written design notes

1. Context

Our client is building an AI-courses platform for the Indian market. While our internal course content is still in production, learners on any AI/ML learning journey share a common, persistent need: **understanding the research and writing that underpins the field**, in a way that is trustworthy, traceable, and tailored to their level. Generic chatbots fail this need — they hallucinate citations, conflate sources, and offer no way to verify a claim against the original work.

This assignment asks you to build the kind of research-and-explanation companion that will eventually sit alongside the courses, and which today is independently useful as a learning aid built on a curated set of high-quality AI/ML documents.

The architecture you build here is intended to transfer directly to the internal course corpus when it becomes available. **Treat the corpus as a variable, not a constant** — your code should remain correct if the document set is swapped wholesale.

2. Problem Statement

Design and implement the **AI Research Navigator**, a Retrieval-Augmented Generation (RAG) system augmented with an agentic decision layer, that helps a learner ask substantive questions about AI/ML and receive answers grounded in a curated set of 50 research papers, surveys, and high-quality educational writings. The system must:

1. **Ingest** the provided corpus into Qdrant, preserving rich metadata that supports precise filtering.
2. **Answer** learner queries using retrieved context, producing responses in which **every substantive claim carries an inline citation** linking to the originating chunk and source — modelled on the citation style used by Perplexity.

3. **Route** queries through a LangGraph-based agent that selects between distinct sub-capabilities: concept explanation, paper deep-dive, approach comparison, recent-developments lookup, paper recommendation, and graceful fallback.
 4. **Refuse gracefully** when retrieval confidence is low, rather than hallucinating an answer or fabricating a citation.
-

3. Scope and Non-Goals

In scope

- Ingestion, retrieval, generation, agent routing, evaluation, and (optionally) a thin UI layer.
- English-language corpus and queries by default. Multilingual handling is a bonus track.

Explicitly out of scope

- Corpus acquisition, sourcing, license tracking, and curation. The corpus is provided as a sealed package.
 - Authentication, billing, and any production-grade infrastructure concerns.
 - Mobile clients.
 - Fine-tuning embedding or generation models. Use off-the-shelf models throughout.
-

4. The Provided Corpus

You will receive a **sealed corpus of 50 documents** along with a pre-populated manifest. The corpus composition:

Content type	Count	Notes
<code>arxiv_paper</code> (foundational)	10	Landmark papers; pre-flagged <code>is_foundational: true</code> .
<code>arxiv_paper</code> (recent)	20	Published within the last 18–24 months.
<code>course_chapter</code>	12	Hugging Face Learn course chapters spanning NLP, Agents, and RL.
<code>survey_blog</code>	5	Long-form AI/ML surveys (Lil'Log).
<code>lab_blog_post</code>	3	Technical research posts from major AI labs.

You will receive each document in its native format (PDF for arXiv papers, Markdown for course chapters and blog posts) along with `corpus/manifest.json` containing the following pre-populated metadata per document:

Field	Type	Provided?
<code>doc_id</code>	string	✓
<code>content_type</code>	enum	✓
<code>title</code>	string	✓
<code>authors</code>	list[string]	✓
<code>year, month</code>	int	✓
<code>primary_category</code>	string	✓ (e.g., <code>cs.CL</code> , or a normalised topic tag for non-arXiv content)
<code>secondary_categories</code>	list[string]	✓
<code>tags</code>	list[string]	✓ (normalised topic tags: <code>LLM</code> , <code>RAG</code> , <code>agents</code> , <code>RLHF</code> , etc.)
<code>is_foundational</code>	bool	✓
<code>citation_count</code>	int (nullable)	✓ for arXiv papers
<code>source_url</code>	string	✓
<code>local_path</code>	string	✓

You are responsible for **deriving and indexing further per-chunk metadata** during ingestion (see M1).

5. Milestones

Milestones are sequenced but not strictly serial. Each has explicit acceptance criteria.

M0 — Project setup and onboarding

- Initialise a clean repository with dependency management via `uv` or `poetry` (locked dependencies, no `pip freeze` dumps).
- Configure pre-commit hooks: `ruff` (lint + format), `mypy` (type checking, strict mode on `src/`), and basic secret scanning.
- Stand up a local Qdrant instance via Docker Compose.
- Skim a representative sample across all five content types in the provided corpus; document initial observations about structural differences and the chunking implications.

Acceptance: `make setup` && `make lint` && `make test` runs cleanly on a fresh clone.

M1 — Ingestion pipeline

1a. Parsing

- Parse each content type appropriately: PDF text extraction for arXiv papers (`pymupdf` or equivalent), Markdown parsing for course chapters and blog posts. Preserve section structure where present — section headings are valuable metadata.
- Handle the practical realities of academic PDFs: multi-column layouts, footnotes, references sections, figures and tables (which you will likely have to skip or summarise). Document your decisions.

1b. Chunking

- Design a chunking strategy with a written rationale. Different content types may warrant different strategies; defend whatever you choose. At a minimum, account for: paper abstracts (treat distinctly), section boundaries, code blocks, and references sections (which should be excluded from retrieval but retained for citation lookup).

1c. Qdrant collection schema

Extend the provided document-level metadata into a per-chunk payload that, at minimum, also includes:

Field	Type	Notes
<code>section_title</code>	string	The section/chapter this chunk belongs to
<code>section_index</code>	int	
<code>chunk_index</code>	int	

Field	Type	Notes
<code>content_hash</code>	string	For change detection on re-ingest

All document-level fields from `manifest.json` (see §4) must be carried through to every chunk.

1d. Idempotent upserts Implement upserts keyed on a deterministic chunk ID derived from (`doc_id`, `chunk_index`, `content_hash`). Re-ingesting an unchanged corpus must be a no-op; changing a document's content must trigger targeted updates only to its chunks.

1e. Payload indexing Make use of Qdrant's payload indexing on the high-cardinality fields you will filter on (`content_type`, `year`, `tags`, `primary_category`, `is_foundational`) to keep filtered queries performant.

1f. CLI Expose `python -m research_navigator.ingest ...` with at minimum: `ingest`, `validate`, `reindex`, `stats`.

Acceptance: Re-running ingestion on an unchanged corpus produces zero writes; modifying a single document triggers updates only to its chunks; `stats` accurately reports counts by `content_type`, `year`, and `tags`.

M2 — Query pipeline with citations

- Implement query understanding: extract intent and inferable metadata filters from the user's question (e.g., "recent work on speculative decoding" → `year >= 2023`, `tags ⊇ ["speculative_decoding", "inference"]`).
- Implement **hybrid retrieval** in Qdrant: dense vectors + sparse (BM25 or SPLADE) with a fusion strategy of your choice. Justify the choice.
- Apply metadata filters at retrieval time using Qdrant's filter primitives — not as a post-hoc step in Python.
- Generate answers in which **every factual sentence carries an inline citation marker** (`[1]`, `[2]`, ...) mapping to a structured citation block. Citations must include: title, authors (first author et al. for ≥ 3 authors), year, source (e.g., "arXiv:2305.14314", "Lil'Log", "Hugging Face Learn"), section, and a direct URL.
- Implement a **low-confidence refusal path**: if top-k retrieval similarity falls below a tuned threshold, return a graceful "I don't have enough relevant material in the corpus to answer this confidently" response rather than fabricating one.
- Implement **citation deduplication**: if multiple retrieved chunks come from the same document, collapse them into a single citation entry that points to the most relevant section.

Acceptance: For a held-out set of 20 questions, every generated answer either carries valid citations to retrieved chunks or refuses; no claim appears without attribution; no citation is fabricated (every cited chunk traces back to a real retrieved chunk).

M3 — Agentic layer with LangGraph

Construct a LangGraph state machine with, at minimum, the following nodes:

- **Router** — classifies the incoming query into one of: `concept_explanation`, `paper_deep_dive`, `compare_approaches`, `recent_developments`, `find_papers`, `out_of_scope`.
- **ConceptExplanation** — invokes the M2 pipeline with a synthesis-oriented prompt; pulls across multiple sources at varying depth.
- **PaperDeepDive** — when the query names or refers to a specific paper, retrieves preferentially from that document's chunks.
- **CompareApproaches** — given two named methods/papers, retrieves from both and produces a structured comparison.
- **RecentDevelopments** — applies a recency filter (e.g., last 12 months) and returns a chronologically ordered digest.
- **FindPapers** — recommends a reading list given a topic and a stated level; relies on metadata filters (`is_foundational`, `citation_count`, `year`) rather than free-form generation.
- **Fallback** — handles `out_of_scope` with a polite decline.

State must be explicit and serialisable. Include at least one tool call within an agent node (e.g., a structured corpus-metadata lookup, or a date-math helper for `RecentDevelopments`).

Acceptance: Each route is exercised by at least three test queries; the graph is visualised (LangGraph's built-in renderer is fine) and included in the design notes.

M4 — Evaluation harness

- Curate a **golden set of ~40 questions** spanning the six routes, with expected source documents (and where feasible, expected sections) for the retrieval subset.
- Implement automated metrics:
 - **Retrieval precision and recall @ k** against expected sources.
 - **Citation faithfulness** via an LLM-as-judge rubric (define the rubric explicitly: does every cited claim follow from the cited source? Are there uncited claims?).
 - **Refusal correctness**: on a held-out set of deliberately out-of-corpus questions, does the system refuse?
 - **Latency** (p50, p95) and **token cost** per query.
- Produce a one-page evaluation report comparing at least two configurations (e.g., dense-only vs. hybrid retrieval; or with vs. without metadata-aware filtering).

Acceptance: `make eval` runs the full harness and emits a structured report (JSON + a Markdown summary).

M5 — Engineering standards (*continuous; gated at every milestone review*)

Non-negotiable and assessed throughout:

- **Code structure:** clear `src/` layout, modular packages (`ingest`, `retrieve`, `generate`, `agents`, `eval`), no business logic in CLI entrypoints.
 - **Typing:** strict `mypy`; no `Any` in public interfaces without justification.
 - **Configuration:** `pydantic-settings` for all config; no hardcoded paths, models, or thresholds.
 - **Logging:** structured logging (`structlog` or equivalent). No `print` statements outside CLI output.
 - **Testing:** unit tests for parsing, chunking, filter inference, citation rendering, and refusal logic; integration tests for the ingestion CLI and the LangGraph router. Minimum 70% coverage on `src/`.
 - **Documentation:** a substantive `README.md`, an `ARCHITECTURE.md`, and short ADRs for each significant choice (chunking strategy per content type, embedding model, hybrid fusion, refusal threshold).
 - **Reproducibility:** fixed seeds where randomness is involved; locked dependency files committed; a Docker Compose file that brings the whole stack up with `docker compose up`.
-

6. Bonus Tracks (*in priority order*)

Do not start a bonus until M0–M4 are solid.

B1 — Query UI

A minimal but polished chat interface (Streamlit, Next.js, or your choice) that:

- Streams responses token-by-token.
- Renders inline citation markers as interactive badges; clicking a citation opens the source chunk with surrounding context and a link to the originating paper/post.
- Preserves conversation history within a session.
- Surfaces the agent's chosen route to the user (transparency).

B2 — Ingestion UI

A web interface that supports:

- Triggering re-ingestion of the corpus or a subset.
- A collection dashboard: chunk counts by `content_type` / `year` / `tags`, recent updates, ability to trigger re-embedding for a subset.
- Inspection of any document's chunks, with their derived metadata.

B3 — Reranking layer

Introduce a cross-encoder reranker between retrieval and generation. Measure the impact on retrieval precision and citation faithfulness in M4 and report it.

B4 — Vernacular query handling

- Detect the query language (Hindi or Tamil) and either translate to English at query time or use a multilingual embedding model end-to-end.
 - Evaluate on a small set of translated questions and report the precision/recall delta vs. English.
-

7. Deliverables

1. **GitHub repository** with full source, tests, Docker setup, and locked dependencies.
 2. **README.md** that gets a reader from clone to a working demo in under five minutes.
 3. **ARCHITECTURE.md** with a system diagram and an explanation of each major design decision.
 4. **ADR folder** with at least five short decision records.
 5. **Evaluation report** (`eval/report.md`) summarising M4 results.
 6. **A 10–15 minute recorded demo** walking through ingestion, queries across all six agent routes, and the evaluation harness.
-

8. How You Will Be Assessed

Dimension	Weight
Correctness of RAG output and citation fidelity	25%
Quality of the ingestion pipeline and metadata design	15%
Agent design and LangGraph implementation	15%

Dimension	Weight
Engineering discipline (M5 standards)	20%
Evaluation rigour and honest reporting of limitations	15%
Design notes, ADRs, and clarity of reasoning	10%
Bonus tracks	up to +15%

We weight engineering discipline heavily and on purpose. A working prototype with sloppy code will score lower than a slightly less feature-complete submission that is clean, tested, and well-reasoned.

9. Ground Rules

- **Ask early, ask often.** Surfacing ambiguity is a strength, not a weakness. Every assumption you make in lieu of asking should be documented explicitly in your design notes.
 - **OSS-first.** Prefer open-source tooling and self-hosted services unless you can justify, in writing, why a proprietary alternative is materially better for the requirement at hand.
 - **No silent failure.** If a component degrades — retrieval returns nothing, an LLM call times out, parsing encounters a malformed PDF — the system must log it clearly and either recover or fail loudly. Errors swallowed in `except: pass` blocks will be treated as bugs.
 - **Honest evaluation.** Report what doesn't work. We are far more interested in a clear-eyed account of limitations than in a polished demo that hides them.
 - **Respect sources.** The corpus consists of others' work. Cite faithfully, link back, and never present a generated paraphrase as if it were the source's own phrasing.
-

10. Suggested Reading

- [Qdrant documentation](#): filtering, payload indexing, hybrid search, sparse vectors.
- [LangGraph documentation](#): state graphs, conditional edges, tool nodes, streaming.
- The original RAG paper (Lewis et al., 2020) and more recent retrieval-evaluation literature (RAGAS, TRIAD, ARES).
- Perplexity's published material on citation rendering and grounding [[Source1](#)].

Good luck. We expect to learn from your work as much as you learn from the assignment.

Data Corpus Details

AI Research Navigator — Curated Corpus (50 documents)

Composition

Content type	Count
<code>arxiv_paper</code> — foundational	10
<code>arxiv_paper</code> — recent (2024–2025)	20
<code>course_chapter</code> (Hugging Face Learn)	12
<code>survey_blog</code> (Lil'Log)	5
<code>lab_blog_post</code>	3
Total	50

Tag vocabulary used throughout (normalised; case-sensitive):

`transformers`, `attention`, `pretraining`, `fine_tuning`, `instruction_tuning`, `prompting`, `chain_of_thought`, `reasoning`, `tool_use`, `LLM`, `MoE`, `RAG`, `retrieval`, `agents`, `alignment`, `RLHF`, `RLAIF`, `DPO`, `preference_optimization`, `safety`, `interpretability`, `efficiency`, `quantization`, `long_context`, `evaluation`, `benchmark`, `multimodal`, `vision_language`, `open_models`, `scaling`, `inference_optimization`, `survey`, `architecture`, `few_shot`, `RL`.

1. Foundational arXiv papers (10)

All marked `is_foundational: true`, `content_type: arxiv_paper`. These define the conceptual backbone of modern LLM-era AI/ML.

#	doc_id	Title	Authors (lead)	Year	Tags	Rationale
1	<code>arxiv-1706.03762</code>	Attention Is All You Need	Vaswani et al.	2017	<code>transformers</code> ,	The Transformer paper.

#	doc_id	Title	Authors (lead)	Year	Tags	Rationale
					attention, architecture	Non-negotiable.
2	arxiv-1810.04805	BERT: Pre-training of Deep Bidirectional Transformers	Devlin et al.	2018	pretraining, transformers, LLM	Foundational encoder pretraining.
3	arxiv-2005.14165	Language Models are Few-Shot Learners (GPT-3)	Brown et al.	2020	LLM, scaling, few_shot	The scaling/few-shot watershed.
4	arxiv-2005.11401	Retrieval-Augmented Generation for Knowledge-Intensive NLP	Lewis et al.	2020	RAG, retrieval	Defines the assignment's core paradigm.
5	arxiv-2106.09685	LoRA: Low-Rank Adaptation of Large Language Models	Hu et al.	2021	fine_tuning, efficiency	Canonical PEFT method.
6	arxiv-2201.11903	Chain-of-Thought Prompting Elicits Reasoning in LLMs	Wei et al.	2022	prompting, chain_of_thought, reasoning	Foundational reasoning prompt technique.
7	arxiv-2203.02155	Training Language Models to Follow	Ouyang et al.	2022	RLHF, alignment	Defines RLHF for

#	doc_id	Title	Authors (lead)	Year	Tags	Rationale
		Instructions (InstructGPT)			t, instruct ion_tuning	instruction-following.
8	arxiv-22 10.03629	ReAct: Synergizing Reasoning and Acting in LMs	Yao et al.	2022	agents, reasoning, tool_use	Foundational agent reasoning pattern.
9	arxiv-22 12.08073	Constitutional AI: Harmlessness from AI Feedback	Bai et al.	2022	alignment, RLAIIF, safety	Introduces RLAIIF.
10	arxiv-23 07.09288	Llama 2: Open Foundation and Fine-Tuned Chat Models	Touvron et al.	2023	LLM, open_models, RLHF	Defining open-weight family; detailed training recipe.

2. Recent arXiv papers (20) — 2024 onward

All `content_type: arxiv_paper`, `is_foundational: false`. Curated for spread across LLM training, RAG, agents, alignment, efficiency, reasoning, multimodal, and evaluation.

#	doc_id	Title	Authors (lead)	Year	Month	Tags
11	arxiv-2401. 04088	Mixtral of Experts	Jiang et al.	2024	1	LLM, MoE, open_models
12	arxiv-2401. 10020	Self-Rewarding Language Models	Yuan et al.	2024	1	alignment, preference_optimization

#	doc_id	Title	Authors (lead)	Year	Month	Tags
13	arxiv-2401.15884	Corrective RAG (CRAG)	Yan et al.	2024	1	RAG, retrieval
14	arxiv-2402.01306	KTO: Model Alignment as Prospect Theoretic Optimization	Ethayarajh et al.	2024	2	alignment, preference_optimization
15	arxiv-2402.17764	The Era of 1-bit LLMs (BitNet b1.58)	Ma et al.	2024	2	quantization, efficiency, inference_optimization
16	arxiv-2403.05530	Gemini 1.5: Unlocking Multimodal Understanding	Google	2024	3	multimodal, long_context, LLM
17	arxiv-2403.04132	Chatbot Arena: An Open Platform for Evaluating LLMs	Chiang et al.	2024	3	evaluation, benchmark
18	arxiv-2403.10131	RAFT: Adapting Language Model to Domain-Specific RAG	Zhang et al.	2024	3	RAG, fine_tuning
19	arxiv-2404.02060	Long-context LLMs Struggle with Long In-context Learning	Li et al.	2024	4	long_context, evaluation

#	doc_id	Title	Authors (lead)	Year	Month	Tags
20	arxiv-2404.14219	Phi-3 Technical Report	Microsoft	2024	4	LLM, open_models, scaling
21	arxiv-2404.16130	From Local to Global: A Graph RAG Approach	Edge et al.	2024	4	RAG, retrieval
22	arxiv-2405.14734	SimPO: Simple Preference Optimization	Meng et al.	2024	5	alignment, preference_optimization
23	arxiv-2405.15793	SWE-Agent: Agent-Computer Interfaces Enable Software Engineering LMs	Yang et al.	2024	5	agents, tool_use
24	arxiv-2406.01574	MMLU-Pro: A More Robust and Challenging Benchmark	Wang et al.	2024	6	evaluation, benchmark
25	arxiv-2407.08608	FlashAttention-3: Fast and Accurate Attention with Asynchrony	Shah et al.	2024	7	attention, efficiency, inference_optimization
26	arxiv-2407.10671	Qwen2 Technical Report	Alibaba	2024	7	LLM, open_models, multilingual

#	doc_id	Title	Authors (lead)	Year	Month	Tags
27	arxiv-2407.21783	The Llama 3 Herd of Models	Meta	2024	7	LLM, open_models, scaling
28	arxiv-2408.00118	Gemma 2: Improving Open Language Models at a Practical Size	Google	2024	8	LLM, open_models (verify exact ID)
29	arxiv-2412.19437	DeepSeek-V3 Technical Report	DeepSeek-AI	2024	12	LLM, MoE, scaling
30	arxiv-2501.12948	DeepSeek-R1: Incentivizing Reasoning Capability via RL	DeepSeek-AI	2025	1	reasoning, RL, LLM

Notes on the recent set:

- LLM/training: 6 (Mixtral, Llama 3, DeepSeek-V3, Qwen2, Phi-3, Gemma 2)
- RAG: 3 (CRAG, RAFT, GraphRAG)
- Agents: 1 (SWE-Agent) — paired with [concept_explanation](#) and [paper_deep_dive](#) opportunities
- Alignment / preference optimisation: 3 (Self-Rewarding, KTO, SimPO) — strong [compare_approaches](#) material
- Efficiency / quantisation: 2 (BitNet b1.58, FlashAttention-3)
- Multimodal / long context: 2 (Gemini 1.5, Long-context Struggle)
- Reasoning: 1 (DeepSeek-R1) — strong contrast with Chain-of-Thought (#6)
- Evaluation: 2 (Chatbot Arena, MMLU-Pro)

Designed [CompareApproaches](#) test cases this set enables:

- DPO (foundational lit) vs. KTO vs. SimPO — three preference-optimisation flavours
- CRAG vs. RAFT vs. GraphRAG — three RAG enhancements
- Llama 3 vs. DeepSeek-V3 vs. Qwen2 — three open-weight training recipes

- CoT (#6) vs. DeepSeek-R1 — explicit reasoning prompts vs. RL-trained reasoning
-

3. Hugging Face Learn course chapters (12)

All `content_type: course_chapter`. Sourced from publicly available HF Learn courses; Markdown sources are available in the [huggingface/course](#) and [huggingface/agents-course](#) repositories on GitHub, which simplifies parsing.

#	doc_id	Course	Chapter	Tags
31	hf-nlp-ch01	NLP Course	Chapter 1 — Transformer models	transformers , architecture , nlp
32	hf-nlp-ch02	NLP Course	Chapter 2 — Using Transformers	transformers , nlp
33	hf-nlp-ch03	NLP Course	Chapter 3 — Fine-tuning a pretrained model	fine_tuning , nlp
34	hf-nlp-ch05	NLP Course	Chapter 5 — The Datasets library	nlp , data
35	hf-nlp-ch06	NLP Course	Chapter 6 — The Tokenizers library	nlp , tokenization
36	hf-nlp-ch07	NLP Course	Chapter 7 — Main NLP tasks	nlp
37	hf-agents-u01	Agents Course	Unit 1 — Introduction to Agents	agents
38	hf-agents-u02	Agents Course	Unit 2 — Frameworks for AI Agents	agents , tool_use
39	hf-agents-u03	Agents Course	Unit 3 — Use Case: Agentic RAG	agents , RAG
40	hf-dr1-u01	Deep RL Course	Unit 1 — Introduction to Deep RL	RL

#	doc_id	Course	Chapter	Tags
41	hf-drl-u04	Deep RL Course	Unit 4 — Policy Gradient methods	RL
42	hf-drl-u08	Deep RL Course	Unit 8 — Proximal Policy Optimization	RL

Note: HF Learn course structures evolve; chapter numbering and unit titles may have shifted slightly since my cutoff. Pull the GitHub source repos and confirm before assembly.

4. Lil'Log surveys (5)

All [content_type: survey_blog](#), [primary_category: survey](#). Selected for breadth and frequent reference in the agents/alignment/safety literature.

#	doc_id	Title	Published	Tags
43	lillog-llm-agents-2023-06	LLM Powered Autonomous Agents	2023-06	agents , survey , tool_use
44	lillog-prompt-eng-2023-03	Prompt Engineering	2023-03	prompting , survey
45	lillog-adversarial-2023-10	Adversarial Attacks on LLMs	2023-10	safety , survey
46	lillog-extrinsic-hallucinations-2024-07	Extrinsic Hallucinations in LLMs	2024-07	safety , evaluation , survey
47	lillog-reward-hacking-2024-11	Reward Hacking in Reinforcement Learning	2024-11	RL , alignment , safety , survey

These are exceptionally useful for the [concept_explanation](#) agent route because each is a long-form synthesis across many primary sources.

5. Lab blog posts (3)

All `content_type: lab_blog_post`. Selected to be technical research posts, not product announcements.

#	doc_id	Lab	Title	Published	Tags
48	<code>anthropic-mapping-mind-2024-05</code>	Anthropic	Mapping the Mind of a Large Language Model	2024-05	<code>interpretability, safety</code>
49	<code>openai-weak-to-strong-2023-12</code>	OpenAI	Weak-to-Strong Generalization	2023-12	<code>alignment, safety</code>
50	<code>deepmind-alphaproof-2024-07</code>	Google DeepMind	AI Achieves Silver-Medal Standard Solving IMO Problems (AlphaProof / AlphaGeometry 2)	2024-07	<code>reasoning, RL, agents</code>

6. Verification checklist

Before assembling the corpus:

- ☐ Confirm every arXiv ID resolves to the intended title on arxiv.org. Two to scrutinise especially: `2408.00118` (Gemma 2) and `2501.12948` (DeepSeek-R1).
- ☐ Download all arXiv PDFs from <https://arxiv.org/pdf/{id}.pdf>. Store under `documents/arxiv/{doc_id}.pdf`.
- ☐ Pull HF Learn chapters from the huggingface/course and huggingface/agents-course GitHub repos (Markdown sources). Confirm chapter slugs match the live site. Store under `documents/hf-learn/{doc_id}.md`.
- ☐ Save Lil'Log posts as Markdown by either pulling from [lilianweng/lilianweng.github.io](https://lilianweng.github.io) (GitHub source) or scraping the live HTML. Store under `documents/lillog/{doc_id}.md`.
- ☐ Save lab blog posts as Markdown (use a tool like [trafilatura](https://trafilatura.com) for clean HTML→Markdown). Store under `documents/lab-blogs/{doc_id}.md`.

- ☐ (Optional) Populate `citation_count` for the arXiv entries via the Semantic Scholar API. Leave `null` if skipped.
 - ☐ Validate `manifest.json` against the schema before handing the corpus to the interns.
-

7. Manifest

A ready-to-use `corpus/manifest.json` accompanies this document. The two should be reviewed together — every entry in the JSON corresponds to a row above, in the same order.